Tributary

Query Kafka topics from DuckDB.

Install & load

```
INSTALL tributary FROM community;
LOAD tributary;
```

01 At a glance

Read Kafka topic messages as rows with topic, partition, offset and BLOB payload columns. Discover cluster metadata, decode application payloads, and materialize a local table for joins and repeatable downstream analysis.

Kafka topic → Message rows → SQL analysis

02 Reads & client configuration

`tributary_scan_topic(topic, …)`

Read a topic as metadata columns and raw message bytes.

`tributary_metadata(…)`

Inspect broker and topic arrays without scanning a topic.

`tributary_version()`

Identify the loaded extension build.

`"bootstrap.servers"`

Named parameter listing Kafka bootstrap host:port addresses.

`"group.id"`

Choose a dedicated consumer group for the analysis.

`"enable.auto.commit"`

Set to 'false' when analytics must not auto-commit offsets.

`"security.protocol"`

Configure PLAINTEXT, SSL or SASL transport for your broker.

03 Keep in mind

Read the payload deliberately

Quote the offset column as "offset". message is a BLOB; decode(message) interprets UTF-8, and JSON extraction requires valid JSON payloads. Broker client options use quoted librdkafka names, including their dots.

04 Discover topics

REQUIRES SETUP

```
SELECT unnest(topics).name AS topic
FROM tributary_metadata(
  "bootstrap.servers" := 'localhost:9092'
);
```

Requires a compatible Tributary build and a Kafka broker on localhost:9092.

05 Snapshot a partition range

REQUIRES SETUP

```
CREATE TABLE topic_snapshot AS
SELECT partition, "offset", message
FROM tributary_scan_topic('events',
  "bootstrap.servers" := 'localhost:9092',
  "group.id" := 'quick-reference',
  "enable.auto.commit" := 'false'
)
WHERE partition = 0
  AND "offset" BETWEEN 0 AND 99;
SELECT "offset",
       decode(message)::JSON->>'user_id' AS id
FROM topic_snapshot;
```

Assumes events contains UTF-8 JSON messages. Add broker authentication when required.

06 In practice

Plan the read

Use a dedicated consumer group and restrict the topic, partition and offset range for ad-hoc work. This is a consumer interface; publishing belongs in a producer. Community package availability varies by DuckDB version and platform.

Full docs & examples ↗ Source ↗